

Κατανεμημένα Συστήματα 2ο Φυλλάδιο εργαστηρίου (04/03/2026)

Java Sockets

Η πιο διαδεδομένη αρχιτεκτονική ανάπτυξης δικτυακών εφαρμογών είναι η αρχιτεκτονική **πελάτη-εξυπηρετητή (client-server)**. Στην αρχιτεκτονική αυτή, ο εξυπηρετητής (server) αποτελεί μια διεργασία που εκτελείται σε έναν υπολογιστή και αναμένει αιτήματα σύνδεσης από προγράμματα-πελάτες (clients), προκειμένου να τους παρέχει συγκεκριμένες υπηρεσίες.

Η Java παρέχει ενσωματωμένη υποστήριξη για ανταλλαγή δεδομένων μεταξύ προγραμμάτων που εκτελούνται σε διαφορετικούς υπολογιστές μέσω της χρήσης **sockets**. Η επικοινωνία πραγματοποιείται στο πλαίσιο της αρχιτεκτονικής πελάτη - εξυπηρετητή, με τη δημιουργία ενός καναλιού επικοινωνίας μεταξύ των δύο άκρων.

Ένα **socket** ορίζεται ως το ένα άκρο ενός αμφίδρομου (bidirectional) καναλιού επικοινωνίας μεταξύ δύο εφαρμογών που εκτελούνται σε δίκτυο. Προσδιορίζεται από: το πρωτόκολλο επικοινωνίας, τη διεύθυνση IP και τον αριθμό θύρας (port number).

Κάθε εφαρμογή διαβάζει και γράφει δεδομένα σε ένα socket μέσω ροών εισόδου και εξόδου (**InputStream** και **OutputStream**). Η διαδικασία αυτή είναι παρόμοια με την ανάγνωση και εγγραφή σε αρχεία, γεγονός που απλοποιεί σημαντικά τον προγραμματισμό δικτυακών εφαρμογών.

Τύποι Sockets

Ανάλογα με το πρωτόκολλο μεταφοράς, διακρίνουμε δύο βασικές κατηγορίες:

1. TCP Sockets (Transmission Control Protocol)

Το TCP είναι πρωτόκολλο προσανατολισμένο στη σύνδεση (connection-oriented). Πριν την ανταλλαγή δεδομένων, απαιτείται η εγκαθίδρυση σύνδεσης μέσω διαδικασίας τριπλής χειραψίας (three-way handshake). Μπορεί να παρομοιαστεί με τηλεφωνική επικοινωνία: πρώτα δημιουργείται σύνδεση μεταξύ των δύο συνομιλητών και στη συνέχεια πραγματοποιείται η ανταλλαγή πληροφοριών.

Χαρακτηριστικά:

- Αξιοπίστη μεταφορά δεδομένων
- Έλεγχος σφαλμάτων
- Διατήρηση σειράς πακέτων
- Μεγαλύτερο κόστος αρχικοποίησης

Στη Java, ο όρος **Socket** αναφέρεται συνήθως στα TCP sockets.

2. UDP Sockets (User Datagram Protocol)

Το UDP είναι πρωτόκολλο χωρίς σύνδεση (connectionless). Δεν απαιτείται εγκαθίδρυση σύνδεσης πριν την αποστολή δεδομένων.

Μπορεί να παρομοιαστεί με ταχυδρομική υπηρεσία: τα πακέτα (datagrams) αποστέλλονται ανεξάρτητα και δεν υπάρχει εγγύηση ότι:

- θα φτάσουν στον προορισμό,
- θα φτάσουν με τη σωστή σειρά,
- δεν θα χαθούν.

Στη Java, τα UDP sockets υλοποιούνται μέσω της κλάσης **DatagramSocket** και των **DatagramPacket**.

Σύγκριση TCP και UDP

Χαρακτηριστικό	TCP	UDP
Σύνδεση	Ναι	Όχι
Αξιοπιστία	Εγγυημένη	Όχι
Ταχύτητα	Μικρότερη	Μεγαλύτερη
Έλεγχος σφαλμάτων	Ναι	Όχι
Χρήση	Web, Email, File transfer	Streaming, DNS, Real-time apps

Η επιλογή μεταξύ TCP και UDP εξαρτάται από τις απαιτήσεις της εφαρμογής.

TCP/IP Socket Programming - Client

Για την υλοποίηση ενός πελάτη (client) σε Java απαιτείται η δημιουργία σύνδεσης προς τον εξυπηρετητή. Για τον σκοπό αυτό πρέπει να γνωρίζουμε:

- Τη **διεύθυνση του εξυπηρετητή** (IP address ή domain name)
- Τον **αριθμό θύρας** (port) στον οποίο αναμένει συνδέσεις

Οι πληροφορίες αυτές δίνονται ως παράμετροι στον κατασκευαστή της κλάσης (*java.net.Socket*):

```
Socket socket = new Socket("example.com", 8080);
```

Αφού τρέξουμε τον δημιουργό της κλάσης, επιτυγχάνεται η σύνδεση μεταξύ των δύο προγραμμάτων.

Ανταλλαγή μηνυμάτων

Στην συνέχεια για να ανταλλάξουμε μηνύματα μεταξύ των δύο προγραμμάτων θα πρέπει να πάρουμε το *InputStream* και *OutputStream* της σύνδεσης με τις μεθόδους *InputStream* **getInputStream()** και *OutputStream* **getOutputStream()** για να λάβουμε και να στείλουμε αντίστοιχα.

```
InputStream in = socket.getInputStream();
OutputStream out = socket.getOutputStream();
```

Οπότε τώρα χρησιμοποιώντας τις μεθόδους που ορίζονται στις κλάσεις *InputStream* και *OutputStream* μπορούμε να ανταλλάξουμε δεδομένα. Τέλος για να κλείσουμε την σύνδεση εκτελούμε την μέθοδο void **close()** της κλάσης *java.net.Socket*.

Χρήσιμες Μέθοδοι της Κλάσης Socket

Επιπλέον στην κλάση *java.net.Socket* ορίζονται διάφορες μέθοδοι που μας δίνουν χρήσιμες πληροφορίες για την σύνδεση. Κάποιες από αυτές είναι:

```
InetAddress getLocalAddress()
InetAddress getInetAddress()
```

οι οποίες επιστρέφουν αντίστοιχα την τοπική και την απομακρυσμένη διεύθυνση της σύνδεσης. Για πληροφορίες σχετικά με τη θύρα σύνδεσης υπάρχουν οι:

```
int getLocalPort()
int getPort()
```

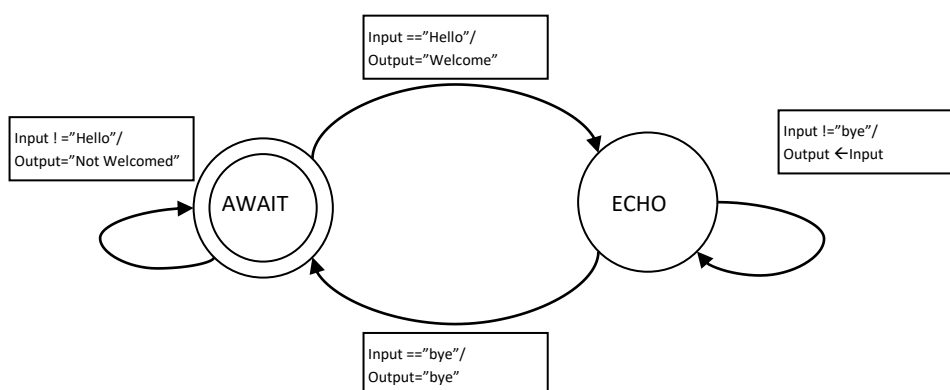
οι οποίες επιστρέφουν την τοπική και την απομακρυσμένη θύρα/πόρτα της σύνδεσης.

Άλλες σημαντικές μέθοδοι της κλάσης Socket είναι:

setSoTimeout()	Η μέθοδος setSoTimeout() επιτρέπει τον ορισμό ενός μεγίστου χρόνου αναμονής για την εγγραφή και ανάγνωση στον εξυπηρετητή. Αν κάτι συμβεί στον εξυπηρετητή ενώ είσαι συνδεδεμένος και υπάρχει σημαντική καθυστέρηση πάνω από το όριο που θέτουμε, τότε θα δημιουργηθεί μία εξαίρεση SocketTimeoutException (setSoTimeout(10000) – 10 sec).
shutdownInput() shutdownOutput()	Η close() μέθοδος διακόπτει την επικοινωνία στο κανάλι, για ανάγνωση αλλά και για εγγραφή. Αν θέλουμε να διακόψουμε είτε την είσοδο είτε την έξοδο τότε μπορούμε να χρησιμοποιήσουμε μία από αυτές τις μεθόδους. Δεν κλείνει το socket αλλά ρυθμίζεται κατάλληλα το stream. (π.χ αποστέλλεται αίτημα σε έναν HTTP Server από τον client και ακολούθως διακόπτεται η εγγραφή).
isInputShutdown() isOutputShutdown()	Για να ελέγξουμε την κατάσταση του stream, αν δηλαδή είναι ενεργή ή όχι η ανάγνωση/εγγραφή.
isClosed()	Επιστρέφει «αληθές» αν η σύνδεση είναι κλειστή.
isConnected()	Δεν ελέγχει αν την τρέχουσα χρονική στιγμή υπάρχει σύνδεση με απομακρυσμένο ΗΥ. Ελέγχει αν έχει πραγματοποιηθεί η σύνδεση και επιστρέφει «αληθές» ακόμα και αν έχει κλείσει το socket. Για να ελέγξουμε αν το socket είναι την τρέχουσα στιγμή ενεργό μπορούμε να εκτελέσουμε τον ακόλουθο έλεγχο: boolean connected = socket.isConnected() && ! socket.isClosed();

1^η Εργαστηριακή Άσκηση – Επικοινωνία Client-Server / Echo Εφαρμογή

Καλείστε να δημιουργήσετε έναν πελάτη που θα συνδέεται σε έναν εξυπηρετητή (localhost) και ο οποίος δέχεται συνδέσεις στην θύρα 5555. Ο εξυπηρετητής υλοποιεί το παρακάτω πρωτόκολλο.



Ενδεικτική έξοδο της εφαρμογής μπορεί να είναι η ακόλουθη:

1η εκτέλεση:

Sending Messages to the Server...

Connecting to /83.212.114.133 and port 5555
Local Address :/195.251.163.201 Port :9442
Write what the client will send: Hello
The server says: Welcome
Write what the client will send: hi
The server says: hi
Write what the client will send: How are you?
The server says: How are you?
Write what the client will send: bye
The server says: bye
Connection Closing...

2η εκτέλεση:

Sending Messages to the Server...
Connecting to /83.212.114.133 and port 5555
Local Address :/195.251.163.201 Port :9429
Write what the client will send: Hi
The server says: Not Welcomed...
Connection Closing...

2^η Εργαστηριακή Άσκηση – Εκτέλεση αριθμητικών πράξεων σε μοντέλο πελάτη-εξυπηρετητή

Καλείστε να δημιουργήσετε ένα πρόγραμμα πελάτη που θα συνδέεται σε έναν εξυπηρετητή (localhost) και ο οποίος δέχεται συνδέσεις στην θύρα 5556. Η εφαρμογή του πελάτη διαβάζει από τον χρήστη 2 ακέραιους αριθμούς και έναν αριθμητικό τελεστή (+, -, * και /). Τα δεδομένα αυτά τα στέλνει στον εξυπηρετητή ο οποίος αναλαμβάνει να εκτελέσει την αριθμητική πράξη και να επιστρέψει το τελικό αποτέλεσμα στον πελάτη. Ο εξυπηρετητής επιστρέφει και ένα μήνυμα εάν ήταν δυνατή η όχι η εκτέλεση της πράξης.

UDP SOCKET PROGRAMMING

Στις προηγούμενες δύο ασκήσεις για την επικοινωνία πελάτη-εξυπηρετητή χρησιμοποιήσαμε TCP sockets. Όπως ήδη αναφέραμε χρησιμοποιώντας το πρωτόκολλο TCP εξασφαλίζουμε την ορθή μετάδοση των πακέτων καθώς και ότι τα πακέτα φθάνουν στη σωστή σειρά, δηλαδή με την σειρά που στάλθηκαν.

Υπάρχουν εφαρμογές στις οποίες δεν μας ενδιαφέρουν τα χαρακτηριστικά αυτά αλλά αντιθέτως μας ενδιαφέρει να επιτύχουμε μεγαλύτερη ταχύτητα στην επικοινωνία. Σε τέτοιες περιπτώσεις είναι καλύτερο να βασιστούμε στο UDP πρωτόκολλο.

Τα datagram πακέτα είναι τα πακέτα που χρησιμοποιούνται για την επικοινωνία πελάτη-εξυπηρετητή με χρήση UDP πρωτοκόλλου. Κάθε μήνυμα μεταφέρεται από την πηγή στον προορισμό, με βάση τις πληροφορίες που εμπεριέχονται σε αυτό το πακέτο.

Η μορφή ενός datagram πακέτου είναι:

Message	Length	Host	Server Port
---------	--------	------	-------------

Η Java υποστηρίζει επικοινωνία με χρήση UDP πρωτοκόλλου μέσω των κλάσεων: DatagramPacket και DatagramSocket.

Η **DatagramPacket** περιέχει διάφορους constructors που μπορούν να χρησιμοποιηθούν για την δημιουργία ενός πακέτου. Ένας τέτοιος constructor είναι ο:

DatagramPacket(byte[] buf, int length, InetAddress address, int port);

Όπως μπορούμε να δούμε από την δήλωση του constructor το μήνυμα που θέλουμε να στείλουμε ορίζεται στην 1^η παράμετρο, το μήκος του μηνύματος στην 2^η παράμετρο ενώ η 3^η και 4^η παράμετρος αναφέρεται στον host (όνομα ή ip διεύθυνση) και στο port που θέλουμε να στείλουμε το μήνυμα. Οι πιο σημαντικές μέθοδοι της κλάσης είναι:

void setData(byte[] buf) και **byte[] getData()**: Για να ορίσουμε ή να λάβουμε αντίστοιχα τα δεδομένα που ανταλλάσσουμε.

void setLength(int length) και **int getLength()**: Για να ορίσουμε ή να λάβουμε το μέγεθος του πακέτου που στέλνουμε ή λαμβάνουμε.

Αντίστοιχα η κλάση **DatagramSocket** περιέχει μεθόδους που μπορούν να χρησιμοποιηθούν για μετάδοση ή λήψη datagram πακέτων, στο δίκτυο. Οι δύο πιο βασικές μέθοδοι είναι οι:

void send(DatagramPacket p): Για την αποστολή ενός πακέτου στο socket.

void receive(DatagramPacket p): Για την λήψη ενός πακέτου από το socket.

3^η Εργαστηριακή Άσκηση – Αποστολή/Λήψη μηνύματος μέσω UDP πρωτοκόλλου

Συμπληρώστε τον κώδικα που ακολουθεί. Το ζητούμενο της άσκησης είναι να δημιουργήσετε έναν πελάτη ο οποίος θα συνδέεται σε έναν εξυπηρετητή (localhost) και ο οποίος δέχεται συνδέσεις στην θύρα 5557. Ο εξυπηρετητής αναμένει αίτημα από τον πελάτη, λαμβάνει το μήνυμα που αποστέλλεται και αμέσως στέλνει ξανά το ίδιο μήνυμα (echo) στον πελάτη. Υλοποιήστε έτσι τον πελάτη ώστε το μήνυμα που θα αποστέλλεται, η διεύθυνση του εξυπηρετητή καθώς και ο αριθμός της πόρτας να ορίζεται με χρήση παραμέτρων προγράμματος.

```
import java.net.*;
import java.io.*;

public class UDPClient {
    public static void main(String args[]){

        DatagramSocket udpSocket = null;
        // Έλεγχος παραμέτρων προγράμματος
        ...

        try {
```

```
udpSocket = new DatagramSocket();

// Διαμόρφωση πακέτου για αποστολή μηνύματος
...

DatagramPacket request = new DatagramPacket(...);

// Αποστολή πακέτου από τον client
...

// Διαμόρφωση πακέτου για λήψη μηνύματος
...

// Λήψη πακέτου από τον server
...

// Εμφάνιση μηνύματος στον client
...
}
catch (SocketException e) {
    System.out.println("Socket: " + e.getMessage());
}
catch (IOException e) {
    System.out.println("IO: " + e.getMessage());
}
finally {
    if (udpSocket != null)
        udpSocket.close();
}
}
```